

Kiến trúc SOA & Microservices

Đặng Ngọc Hùng

Mục lục

8. Cổng API — API Gateway & BFF Pattern	4
8.1. Động lực sử dụng API Gateway	5
8.1.1. Vấn đề: client giao tiếp trực tiếp với nhiều services	5
8.1.2. API Gateway pattern	6
8.1.3. API Gateway vs BFF (Backend for Frontend)	6
8.2. Spring Cloud Gateway — Reactive Gateway	8
8.2.1. Vấn đề: chọn gateway technology	8
8.2.2. Kiến trúc Spring Cloud Gateway	8
8.3. Route Configuration với Eureka	9
8.3.1. Vấn đề: routes hardcoded hay dynamic?	9
8.3.2. LMS Gateway Route Configuration	10
8.3.3. Cách <code>lb://</code> hoạt động	10
8.4. Cross-Cutting Concerns tại Gateway	11
8.4.1. Vấn đề: mỗi service tự xử lý authentication, CORS, logging	11
8.4.2. Authentication — JWT Validation tại Gateway	12
8.4.3. CORS — Cross-Origin Resource Sharing	14
8.4.4. Rate Limiting	15
8.4.5. Logging & Tracing	17
8.5. Case Study: KBM	18
8.5.1. Kiến trúc tổng thể	18
8.5.2. Phân tích configuration	19
8.5.3. Vấn đề JWT Version Inconsistency	20
8.5.4. Đề xuất migration	20
8.6. Tổng kết	22
8.7. Đọc thêm	22
Tài liệu tham khảo	23

8.

CHƯƠNG 8.

Cổng API — API Gateway & BFF Pattern

“The API Gateway is the single entry point for all clients. It handles cross-cutting concerns so that individual services don’t have to.”

— Chris Richardson, *Microservices Patterns* [2a]

MỤC TIÊU HỌC TẬP

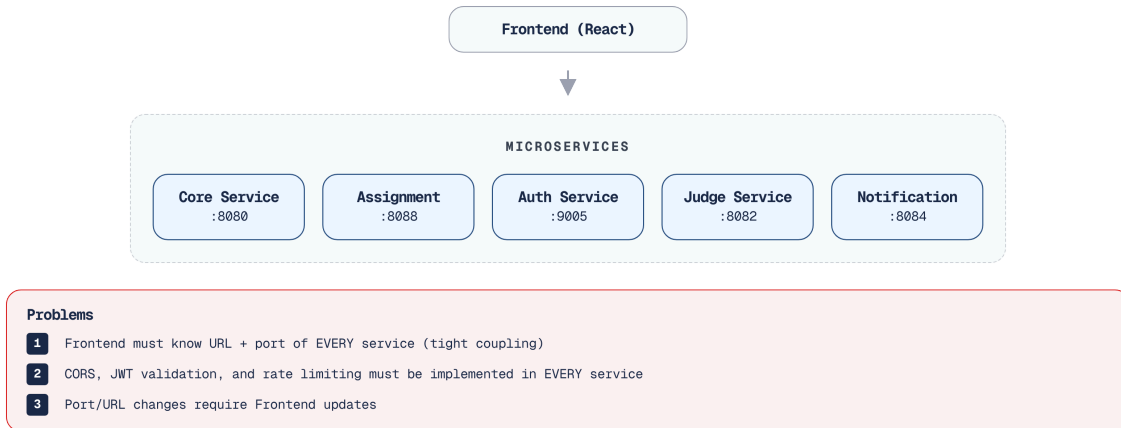
- Hiểu tại sao microservices cần API Gateway và các vấn đề nó giải quyết
- Nắm vững API Gateway pattern và Backend for Frontend (BFF) pattern
- Sử dụng Spring Cloud Gateway (WebFlux) để triển khai gateway
- Cấu hình route với Eureka service discovery (load-balanced URIs)
- Thiết kế cross-cutting concerns tại gateway: authentication, CORS, rate limiting
- Phân tích kiến trúc gateway trong KBM

8.1. Động lực sử dụng API Gateway

Trong một khuôn viên trường đại học rộng lớn, nếu không có một cổng thông tin duy nhất, việc tìm kiếm và truy cập các dịch vụ sẽ trở nên vô cùng rối rắm. Hệ thống microservices cũng đối mặt với một thách thức tương tự ở phía client.

8.1.1. Vấn đề: client giao tiếp trực tiếp với nhiều services

Khi không có gateway, client (web, mobile) phải biết địa chỉ của *từng* microservice và gọi trực tiếp. Với KBM gồm nhiều services và module lab, mỗi trang web có thể cần gọi nhiều API khác nhau:



Hình 8.1: Không có Gateway — client phải biết địa chỉ của từng service

Việc bắt client (đặc biệt là giao diện người dùng) phải tự ghi nhớ và gọi đến từng service riêng lẻ chẳng khác nào yêu cầu tân sinh viên tự cầm bản đồ đi tìm từng phòng ban để hoàn tất thủ tục nhập học thay vì chỉ đến bộ phận “Một cửa” duy nhất.

🔗 Cổng bảo vệ trường Đại học

Hãy tưởng tượng hệ thống Microservices như một khuôn viên trường đại học với hàng chục tòa nhà (Khoa CNTT, Thư viện, Ký túc xá). Nếu không có cổng chính (API Gateway), một khách mời sẽ phải tự cầm bản đồ, tự tìm kiếm và di chuyển đến từng tòa nhà, gõ cửa hỏi đường, và tòa nhà nào cũng phải bố trí bảo vệ riêng để kiểm tra thẻ. **API Gateway** hoạt động như Bộ phận một cửa của trường đại học. Sinh viên chỉ cần đến một địa chỉ duy nhất (một URL), nhân viên trực sẽ kiểm tra thẻ sinh viên (Authentication) và tiếp nhận hoặc chuyển yêu cầu đến đúng phòng ban chức năng (Routing). Các phòng ban bên trong không cần lặp lại bước kiểm tra thẻ nữa.

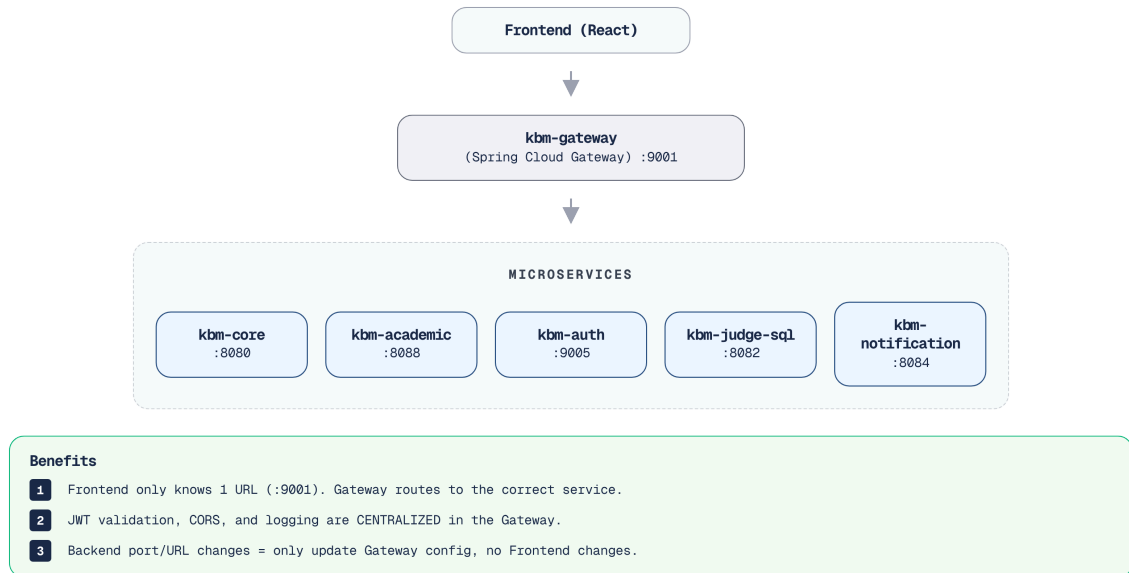
Richardson trong [2a, Ch.8] chỉ ra năm vấn đề khi để ứng dụng client gọi trực tiếp đến từng microservice:

1. **Nhiều endpoints:** Client phải quản lý và ghi nhớ URL của mọi dịch vụ riêng lẻ, dẫn đến sự liên kết chặt chẽ (tight coupling) giữa frontend và backend.
2. **Giao thức khác nhau:** Các dịch vụ có thể sử dụng đa dạng giao thức (REST, gRPC, WebSocket), buộc frontend phải trang bị nhiều thư viện hỗ trợ tương ứng, làm tăng độ phức tạp của mã nguồn.
3. **Cross-cutting concerns phân tán:** Mỗi dịch vụ phải tự triển khai các nghiệp vụ cắt ngang như xác thực, CORS, và rate limiting, gây ra sự trùng lặp và nguy cơ bất nhất quán trong các chính sách hệ thống.
4. **Network không an toàn:** Việc phơi bày trực tiếp toàn bộ các điểm cuối API nội bộ ra môi trường Internet làm gia tăng bề mặt tấn công (attack surface), tạo cơ hội cho các rủi ro bảo mật tiềm ẩn.

5. **API không phù hợp:** Các giao diện lập trình nội bộ thường được thiết kế cho quá trình tương tác giữa máy chủ với máy chủ, trả về lượng dữ liệu dư thừa hoặc yêu cầu quá nhiều vòng gọi mạng (calls), gây nghẽn cổ chai đối với các nền tảng di động có băng thông hạn hẹp.

8.1.2. API Gateway pattern

API Gateway là **single entry point** — toàn bộ yêu cầu (request) từ ứng dụng khách sẽ đi qua cổng giao tiếp (gateway) và được định tuyến (route) đến đúng dịch vụ tương ứng:



Hình 8.2: API Gateway — single entry point route đến từng service

Gateway xử lý các nghiệp vụ cắt ngang (**cross-cutting concerns**) một cách tập trung, bao gồm: xác thực (authentication), CORS, giới hạn lưu lượng (rate limiting), logging và kết thúc SSL. Các dịch vụ phía sau chỉ tập trung vào business logic — không cần phải tự xử lý các vấn đề liên quan đến CORS.

Gateway thường được ví như một “smart edge” (điểm biên thông minh) chuyên xử lý các cross-cutting concerns. Tuy nhiên, theo đúng tinh thần của nguyên lý “**Smart endpoints and dumb pipes**” do Martin Fowler và Sam Newman đề xuất, Gateway tuyệt đối không được phép chứa business logic. Đối với dữ liệu nghiệp vụ, Gateway vẫn phải duy trì vai trò của một “dumb pipe” (chỉ định tuyến cơ bản) để tránh lặp lại vấn đề của ESB.

8.1.3. API Gateway vs BFF (Backend for Frontend)

Richardson trong [2a, Ch.8] phân biệt hai biến thể:



Hình 8.3: API Gateway (một gateway chung) vs BFF (gateway riêng cho từng client)

Richardson phân biệt hai biến thể chính của cổng giao tiếp. Biến thể phổ thông nhất là **API Gateway**, nơi một cổng duy nhất phục vụ toàn bộ các loại client. Phương pháp này đặc biệt phù hợp với các đội ngũ nhỏ hoặc khi các ứng dụng client (như web admin và web sinh viên) có nhu cầu truy xuất dữ liệu tương đồng nhau. Tuy nhiên, khi hệ thống mở rộng, biến thể **BFF (Backend for Frontend)** lại tỏ ra phù hợp hơn trong kịch bản này bằng cách cung cấp một cổng giao tiếp chuyên biệt cho từng loại client. BFF là giải pháp bắt buộc khi ứng dụng di động (mobile app) đòi hỏi định dạng API hoàn toàn khác biệt so với nền tảng web, ví dụ như cần gộp nhiều lời gọi (batched calls) và tối giản hóa dữ liệu trả về để tiết kiệm băng thông.

KBM sử dụng **một API Gateway duy nhất** cho phân hệ lõi của LMS — điều này hoàn toàn phù hợp vì các web frontend chính đều có chung những yêu cầu API tương tự nhau. Với DevOps Lab, KBM bổ sung một lớp router riêng theo hostname để điều hướng workspace/lab runtime; đây là gateway chuyên biệt cho một bounded context, không thay thế Spring Cloud Gateway của LMS core.

Khi nào cần BFF? BFF trở nên cần thiết khi các client khác nhau có nhu cầu API hoàn toàn khác biệt về bản chất — không chỉ đơn thuần là “lọc bỏ bớt các trường dữ liệu”:

Để quyết định khi nào cần áp dụng BFF, ta có thể xét qua ba kịch bản thực tế. Nếu hệ thống chỉ phục vụ Web và Web Admin với nhu cầu dữ liệu tương tự nhau, một **Single Gateway** là hoàn toàn đủ, việc cố áp dụng BFF ở giai đoạn này sẽ dẫn đến sự dư thừa về mặt kỹ thuật (over-engineering). Khi hệ thống bắt đầu phục vụ thêm ứng dụng di động (Web + Mobile), Single Gateway sẽ bộc lộ điểm yếu khi ép ứng dụng di động phải gọi nhiều API rời rạc; lúc này, một Mobile BFF làm nhiệm vụ gom nhóm dữ liệu (aggregate) là giải pháp phù hợp. Ở kịch bản phức tạp nhất bao gồm Web, thiết bị IoT (như hệ thống điểm danh khuôn mặt ở giảng đường) và đối tác thứ ba (như cổng thanh toán học phí của ngân hàng), việc sử dụng một cổng duy nhất sẽ biến Gateway thành một hệ thống phức tạp, khó bảo trì; đây là lúc hệ thống bắt buộc phải chia nhỏ thành **nhiều BFF độc lập** phục vụ riêng cho từng nhóm client.

Ví dụ: nếu KBM phát triển thêm **mobile app** cho sinh viên, nền tảng này sẽ yêu cầu: (1) **batched API** — giao diện “Bảng điều khiển” (Dashboard) chỉ cần một lời gọi để lấy cả thông tin cá nhân, bài nộp gần đây và thứ hạng (thay vì phải gọi ba lần, giúp khắc phục hạn chế mạng di động chậm); (2) **reduced payload** — thiết bị di động không cần dữ liệu định dạng sẵn cho HTML mà chỉ cần dữ liệu thô gọn nhẹ; (3) **push notification integration** — cổng giao tiếp dành riêng cho di động sẽ quản lý các mã định danh thiết bị (device tokens).

Newman trong [4a, Ch.4] khuyến nghị: BFF nên được **quản lý bởi đội ngũ phát triển giao diện (frontend)** — đội ngũ phát triển ứng dụng di động chịu trách nhiệm về Mobile BFF, đội ngũ phát triển

web đảm nhận Web BFF. Mỗi BFF hoạt động như một lớp xử lý mỏng (thin layer): tiếp nhận yêu cầu từ ứng dụng khách, gọi các dịch vụ nội bộ (downstream services), tiến hành gom nhóm và biến đổi dữ liệu (aggregate và transform), sau đó trả về định dạng phù hợp cho từng loại ứng dụng khách.

⚠ Aggregation vs Orchestration

BFF được phép thực hiện **Data Aggregation** (gom nhiều lời gọi API lại để hiển thị lên giao diện người dùng (UI) một cách nhanh chóng hơn). Tuy nhiên, cần đặc biệt tránh việc sử dụng BFF hay Gateway để thực hiện **Business Orchestration** (điều phối nghiệp vụ, ví dụ: tạo bài nộp (Submission) rồi cập nhật điểm số (Grade)). Mọi logic điều phối giao dịch phải nằm ở tầng Domain Services (sử dụng Saga/Process Manager như ở Chương 6).

8.2. Spring Cloud Gateway – Reactive Gateway

Khi xây dựng API Gateway, việc lựa chọn công nghệ lõi quyết định khả năng mở rộng và chịu tải của hệ thống. Bài toán đặt ra là cần một kiến trúc đáp ứng được lưu lượng kết nối lớn.

8.2.1. Vấn đề: chọn gateway technology

Trong hệ sinh thái Spring, các nhà phát triển thường cân nhắc giữa hai lựa chọn chính khi xây dựng cổng giao tiếp tập trung:

Mô hình hoạt động – Spring Cloud Gateway sử dụng mô hình phản ứng (Reactive - WebFlux, non-blocking), cho phép xử lý số lượng lớn các kết nối đồng thời với rất ít luồng (threads). Trong khi đó, Netflix Zuul (phiên bản 1.x) dựa trên mô hình Servlet truyền thống (blocking, thread-per-request), dễ bị thắt cổ chai khi đối mặt với tải trọng lớn.

Hiệu năng và WebSocket – Nhờ kiến trúc WebFlux, Spring Cloud Gateway mang lại hiệu năng cao hơn và hỗ trợ giao thức WebSocket một cách tự nhiên (native support) – một tính năng cốt lõi cho các ứng dụng hệ thống thời gian thực. Ngược lại, Zuul 1.x có hiệu suất thấp hơn do phụ thuộc vào giới hạn của bộ đệm luồng và hoàn toàn không cung cấp sự hỗ trợ cho WebSocket.

Hệ sinh thái và Tình trạng – Spring Cloud Gateway hiện là dự án đang được tích cực phát triển (active), được Spring chính thức khuyến khích và gắn kết chặt chẽ vào hệ sinh thái Spring Cloud. Trái lại, Netflix Zuul 1.x đã bị đánh dấu lỗi thời (deprecated) do Netflix ngừng bảo trì và hiện chỉ được xem như một hệ thống công nghệ cũ (legacy).

Dựa trên các ưu điểm này, kiến trúc cốt lõi của LMS sử dụng **Spring Cloud Gateway**. Quyết định này nhằm đáp ứng yêu cầu hỗ trợ giao thức WebSocket cho tính năng đẩy thông báo (notification push) liên tục, đồng thời đảm bảo hiệu năng cao cho hệ thống.

8.2.2. Kiến trúc Spring Cloud Gateway

Để hiểu được khả năng xử lý của Spring Cloud Gateway, chúng ta cần phân tích sâu các thành phần cấu trúc cốt lõi định hình nên năng lực định tuyến của nó. Khác với các hệ thống gateway cũ nguyên khối, giải pháp của Spring phân chia quá trình xử lý yêu cầu thành những khái niệm độc lập và dễ dàng kết hợp. 8.4 minh họa mô hình tổng thể của kiến trúc này, nơi một dòng chảy dữ liệu liên tục được lọc, kiểm tra, và quyết định bên đổ cuối cùng dựa trên các tập hợp quy tắc Vị từ (Predicates) và Bộ lọc (Filters).



Hình 8.4: Kiến trúc Spring Cloud Gateway — Predicates, Filters, Routes

Kiến trúc của Spring Cloud Gateway xoay quanh ba khái niệm cốt lõi. Khái niệm cơ bản nhất là **Route** (Tuyến đường), định nghĩa quy tắc định tuyến từ một điều kiện đầu vào đến địa chỉ URI đích (ví dụ: chuyển hướng toàn bộ `/api/core/**` sang `lb://kbm-core`). Để quyết định một request có khớp với Route hay không, hệ thống sử dụng các **Predicate** (Vị từ) như `Path` hay `Method` để thực hiện phép thử boolean. Cuối cùng, để can thiệp vào vòng đời của request và response, các **Filter** (Bộ lọc) được chèn vào trước hoặc sau quá trình định tuyến nhằm xử lý các nghiệp vụ cắt ngang như xác thực thông qua `JwtRequestFilter` hay gắn thêm các header cần thiết.

Về dependency: Gateway sử dụng thư viện `spring-cloud-starter-gateway` (WebFlux) và `spring-cloud-starter-netflix-eureka-client`. **Lưu ý:** Do Gateway hoạt động dựa trên cơ chế WebFlux (mô hình phản ứng, không chặn luồng), nó *không thể* tương thích với `spring-boot-starter-web` (mô hình servlet, chặn luồng). Việc khai báo đồng thời cả hai thư viện này trong dự án Gateway sẽ sinh ra xung đột (conflict) và khiến ứng dụng không thể khởi động.

8.3. Route Configuration với Eureka

Khác với các phòng ban cố định trong trường học, các máy chủ dịch vụ ảo có thể thay đổi địa chỉ liên tục. Việc định tuyến cố định sẽ bộc lộ nhiều hạn chế khi hệ thống bắt đầu tự động thay đổi quy mô.

8.3.1. Vấn đề: routes hardcoded hay dynamic?

Phương pháp đơn giản nhất là gắn cố định (hardcode) địa chỉ URL cho từng dịch vụ trong tệp cấu hình của Gateway. Tuy nhiên, khi các dịch vụ mở rộng quy mô (scale) hoặc được triển khai (deploy) sang vùng chứa mới, các địa chỉ URL sẽ thay đổi. Khi đó, việc cập nhật thủ công cấu hình Gateway sẽ trở nên khó khăn và tiềm ẩn nhiều rủi ro phát sinh lỗi.

Giải pháp: kết hợp định tuyến gateway với **Eureka service discovery** (đã học ở Ch.4). Gateway không cần lưu trữ địa chỉ IP và cổng cụ thể của dịch vụ — nó chỉ cần tên dịch vụ, và Eureka sẽ tự động đảm nhiệm quá trình phân giải và định tuyến.

8.3.2. LMS Gateway Route Configuration

```
# application-lb.yml — LMS Gateway routing configuration
spring:
  cloud:
    gateway:
      routes:
        # Core Service — questions, submissions, contests
        - id: kbm-core
          uri: lb://kbm-core      # lb:// = Eureka lookup + load balance
          predicates:
            - Path=/api/core/**
          filters:
            - StripPrefix=2      # /api/core/questions → /questions

        # Assignment Service — courses, grades
        - id: kbm-academic
          uri: lb://kbm-academic
          predicates:
            - Path=/api/assignment/**
          filters:
            - StripPrefix=2

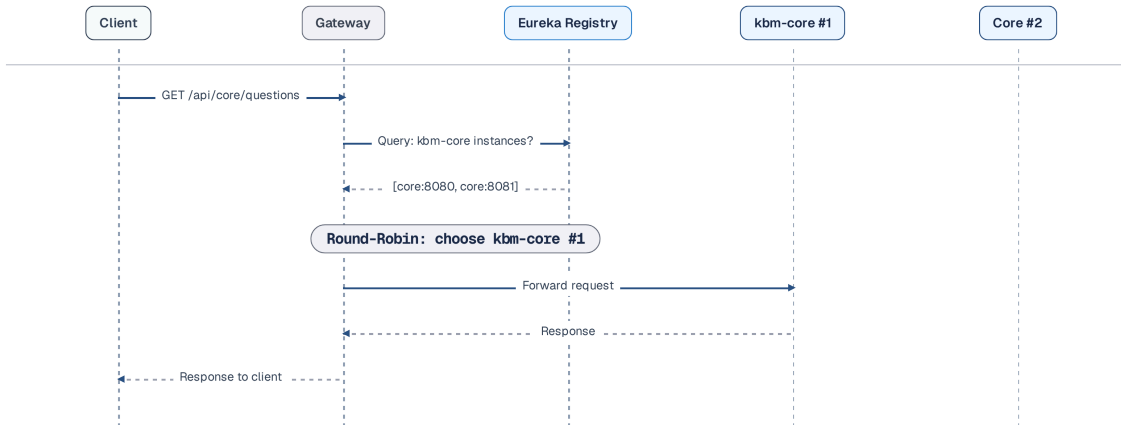
        # Auth Service — login, register
        - id: kbm-auth
          uri: lb://kbm-auth
          predicates:
            - Path=/api/auth/**
          filters:
            - StripPrefix=2

        # Notification — WebSocket endpoint
        - id: kbm-notification-ws
          uri: lb://kbm-notification
          predicates:
            - Path=/ws/**
```

Listing 8.1: LMS Gateway route configuration (YAML + Eureka)

8.3.3. Cách `lb://` hoạt động

8.5 minh họa luồng phân giải chuỗi URI với tiền tố `lb://` trong thực tế, khi một request định tuyến đến một dịch vụ ảo thông qua Eureka thay vì IP tĩnh:



Hình 8.5: Luồng **lb://** — Eureka lookup + load balance + route

lb://kbm-core thực hiện ba bước tự động:

1. **Lookup**: truy vấn (query) Eureka để tìm kiếm toàn bộ các thực thể dịch vụ (instances) có tên **kbm-core**
2. **Load balance**: lựa chọn một thực thể thông qua Spring Cloud LoadBalancer (round-robin mặc định)
3. **Forward**: chuyển tiếp yêu cầu (request) đến thực thể đã được chọn

StripPrefix=2 loại bỏ 2 phần đầu tiên của path: **/api/core/questions** → **/questions**. Nhờ đó, dịch vụ không cần quan tâm đến đường dẫn gốc (mount path) của nó tại Gateway — dịch vụ chỉ cần xử lý đường dẫn nội bộ của chính nó.

Service không biết Gateway

Các dịch vụ phía sau *không nên biết* về sự tồn tại của Gateway. Mỗi dịch vụ tự xử lý (handle) các đường dẫn nội bộ tương ứng (**/questions** , **/users**), trong khi Gateway đảm nhiệm việc gắn tiền tố (prefix) và định tuyến. Điều này đảm bảo: (1) Các dịch vụ có thể được kiểm thử độc lập (standalone) mà không phụ thuộc vào Gateway; (2) Việc cấu trúc lại định tuyến tại Gateway không làm ảnh hưởng đến mã nguồn của dịch vụ; (3) Dịch vụ có tính di động cao (portable) — cho phép dễ dàng triển khai (deploy) sau các cổng giao tiếp khác (như NGINX, Kong) mà không cần chỉnh sửa mã nguồn.

8.4. Cross-Cutting Concerns tại Gateway

Việc triển khai các nghiệp vụ cắt ngang ở mỗi dịch vụ giống như việc từng lớp học tự đặt ra quy định kiểm tra thẻ sinh viên riêng lẻ thay vì giao cho bảo vệ cổng trường: vừa dư thừa, vừa không thống nhất và khó quản lý.

8.4.1. Vấn đề: mỗi service tự xử lý authentication, CORS, logging

Nếu mỗi dịch vụ tự thực hiện xác thực token JWT, tự cấu hình CORS, tự triển khai rate limiting — mã nguồn sẽ bị trùng lặp ở nhiều dịch vụ, và mỗi khi có sự thay đổi về chính sách bảo mật, lập trình viên buộc phải cập nhật mã ở tất cả các dịch vụ đó. Đây chính là vấn đề gateway giải quyết: **tập trung cross-cutting concerns tại một điểm duy nhất**.

8.4.2. Authentication — JWT Validation tại Gateway

💡 Hạn chế của Session Cookie

Trong Monolith, trường đại học chỉ có một cô Thủ thư duy nhất. Khi một sinh viên vào thư viện, cô nhận ra mặt và nhớ đó là sinh viên (Session Cookie). Nhưng khi trường mở rộng ra 10 chi nhánh thư viện (Microservices), sinh viên sang chi nhánh khác, cô thủ thư mới sẽ không biết người đó là ai. Sinh viên bắt buộc phải dùng chung một sổ đăng ký trung tâm (Shared Redis) gây nghẽn thắt cổ chai. Đó là lý do ta chuyển sang **JWT (JSON Web Token)** — hãy coi nó như **Thẻ Sinh Viên Điện Tử**. Trên thẻ JWT đã dán ảnh, ghi mã SV, và có “con dấu đỏ” chữ ký điện tử của phòng Đào tạo. Các tòa nhà (Downstream services) hoặc Cổng bảo vệ (Gateway) chỉ cần nhìn con dấu là biết thẻ thật, hoàn toàn không cần gọi điện hỏi lại phòng Đào tạo (Stateless Validation).

Do hạn chế của Session Cookie trong kiến trúc phân tán, KBM lựa chọn cơ chế xác thực không trạng thái (stateless) với JSON Web Token (JWT). Thay vì để mỗi dịch vụ tự xác minh, hệ thống xử lý tập trung tại Gateway thông qua một `GatewayFilter` tùy chỉnh. Bộ lọc này hoạt động như chốt chặn an ninh đầu tiên, đảm bảo mọi yêu cầu đều có token hợp lệ. Đoạn mã dưới đây minh họa `JwtAuthGlobalFilter` dùng để kiểm chứng yêu cầu và loại bỏ các header nhạy cảm nhằm phòng ngừa giả mạo (header spoofing):

```

// Gateway JWT Filter – validate token trước khi route
@Component
public class JwtAuthGlobalFilter implements GlobalFilter, Ordered {

    private final JwtUtil jwtUtil;

    @Override
    public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain) {
        ServerHttpRequest request = exchange.getRequest();
        String path = request.getURI().getPath();

        // Bắt buộc XÓA các tiêu đề (headers) nhạy cảm nếu ứng dụng khách cố ý gửi kèm (Header Spoofing)
        ServerHttpRequest.Builder requestBuilder = request.mutate()
            .headers(headers -> {
                headers.remove("X-User-Id");
                headers.remove("X-User-Roles");
                headers.remove(org.springframework.http.HttpHeaders.AUTHORIZATION);
            });

        // Bỏ qua xác thực cho các điểm cuối (endpoints) công khai
        if (isPublicEndpoint(path)) {
            return chain.filter(
                exchange.mutate().request(requestBuilder.build()).build()
            );
        }

        // Extract JWT từ Authorization header
        String token = extractToken(request);
        if (token == null || !jwtUtil.validateToken(token)) {
            exchange.getResponse().setStatusCode(HttpStatus.UNAUTHORIZED);
            return exchange.getResponse().setComplete();
        }

        // Ghi đè user info an toàn (Overwrite)
        requestBuilder.headers(headers -> {
            headers.set("X-User-Id", jwtUtil.getUserId(token));
            headers.set("X-User-Roles", jwtUtil.getRoles(token));
        });

        return chain.filter(
            exchange.mutate().request(requestBuilder.build()).build()
        );
    }

    @Override
    public int getOrder() {
        return -1; // Chạy TRƯỚC RequestRateLimiter
    }
}

```

Listing 8.2: Global JWT Filter bảo mật tại Gateway

Sau khi Gateway xác thực JWT, thông tin định danh (claims) cần được truyền an toàn đến các dịch vụ phía sau (downstream). 8.6 mô tả cơ chế luân chuyển các ‘claims’ này thông qua các tiêu đề HTTP (headers) đã được Gateway xác minh:



Hình 8.6: Luồng JWT validation tại Gateway — claims propagation qua trusted headers

Các dịch vụ phía sau nhận thông tin người dùng thông qua các **custom headers** (`X-User-Id` , `X-User-Roles`) — và không cần phải xác thực lại JWT. Đây là pattern **claims-based identity propagation**: Gateway chịu trách nhiệm xác thực token, và các dịch vụ hoàn toàn tin tưởng Gateway (do đây là lưu lượng mạng nội bộ - internal traffic).

⚠ Giả định Network Trust

Mô hình này chỉ an toàn khi **toàn bộ traffic bắt buộc đi qua gateway** — được enforce bằng network policy (firewall rules, Kubernetes NetworkPolicy). Nếu service bị truy cập trực tiếp từ bên ngoài (vượt qua cổng giao tiếp - bypass gateway), header `X-User-Id` / `X-User-Roles` sẽ dễ dàng bị giả mạo. Tuy nhiên, để chống giả mạo tuyệt đối ngay cả khi mạng nội bộ bị hacker xâm nhập, các hệ thống hiện đại thường áp dụng **mTLS** (Mutual TLS) thông qua Service Mesh (như Istio) để mã hóa và xác thực chữ ký của mọi cuộc gọi giữa Gateway và các services.

8.4.3. CORS — Cross-Origin Resource Sharing

Cấu hình CORS tại Gateway đóng vai trò là **một nơi duy nhất** quản lý các nguồn gốc (origins), phương thức (methods) và tiêu đề (headers). Cấu hình `globalcors` trong Spring Cloud Gateway cho phép khai báo `allowedOrigins` (domains hợp lệ), `allowedMethods` , `allowCredentials` — các dịch vụ phía sau không cần thiết lập cấu hình CORS vì Gateway đã đảm nhận chức năng này.

```

spring:
  cloud:
    gateway:
      globalcors:
        cors-configurations:
          '[/**]':
            allowed-origins:
              - "https://kbm.vn"
              - "https://admin.kbm.vn"
            allowed-methods: "GET, POST, PUT, DELETE, OPTIONS"
            allowCredentials: true

```

Listing 8.3: Cấu hình CORS an toàn tại Gateway

Cấu hình CORS ở cấp độ Gateway giúp hệ thống tránh phải thiết lập lại các quy tắc tương tự ở từng dịch vụ. Bằng cách định nghĩa tập trung các nguồn gốc hợp lệ (`allowedOrigins`), phương thức cho phép (`allowedMethods`), và khả năng chấp nhận xác thực (`allowCredentials`), Gateway kiểm soát các yêu cầu truy cập từ miền khác một cách nhất quán. Dù vậy, cấu hình CORS cần được thiết lập cẩn thận để tránh vô tình tạo ra lỗ hổng bảo mật.

i KBM CORS

Hệ thống KBM từng sử dụng cấu hình `allowedOrigins: "*" —` cho phép *mọi* nguồn gốc (origin) truy xuất API. Trong giai đoạn phát triển (development), đây là giải pháp tiện lợi để tránh lỗi CORS. Tuy nhiên, trên môi trường thực tế (production), đây là một **lỗ hổng bảo mật nghiêm trọng**: bất kỳ trang web độc hại nào cũng có thể lợi dụng thông tin xác thực (credentials) của người dùng để gọi API (tấn công CSRF). **Migration:** (1) Liệt kê cụ thể các nguồn gốc hợp lệ dựa trên từng môi trường, không public domain cụ thể trong sách, (2) Cấu hình `allowCredentials: true` đối với cơ chế xác thực dựa trên cookie, (3) Kiểm thử kỹ lưỡng với ứng dụng di động nếu có.

8.4.4. Rate Limiting

Cơ chế giới hạn lưu lượng (Rate limiting) là kỹ thuật để ngăn chặn tình trạng một client gửi quá nhiều yêu cầu trong thời gian ngắn, qua đó bảo vệ các dịch vụ cốt lõi khỏi nguy cơ quá tải hoặc bị tấn công từ chối dịch vụ (DoS). Spring Cloud Gateway hỗ trợ sẵn bộ lọc `RequestRateLimiter` kết hợp với Redis để hiện thực hóa tính năng này. Bằng cách tùy chỉnh các thông số như tỷ lệ phục hồi `replenishRate` (số yêu cầu mỗi giây), dung lượng bùng nổ `burstCapacity` (số lượng yêu cầu tối đa xử lý tức thời), và thành phần xác định khóa `KeyResolver`, hệ thống có thể kiểm soát lưu lượng truy cập dựa trên nhiều chính sách khác nhau. Các chiến lược phổ biến bao gồm:

1. **Per user:** Giới hạn hạn ngạch (ví dụ: N requests/giây) cho từng người dùng đã xác thực. Phương pháp này giúp ngăn chặn lạm dụng hoặc khai thác tài nguyên quá mức từ một tài khoản.
2. **Per IP:** Giới hạn lưu lượng từ một địa chỉ IP duy nhất. Chiến lược này hiệu quả trong việc ngăn chặn tấn công DDoS thô sơ hoặc ngăn chặn các bot cào dữ liệu (scrapping).
3. **Per route:** Thiết lập giới hạn riêng cho từng API. Kỹ thuật này thường áp dụng để bảo vệ các API tiêu tốn nhiều năng lực tính toán hoặc I/O (ví dụ: xuất báo cáo thống kê).
4. **Global:** Giới hạn tổng lượng yêu cầu truy cập đi vào hệ thống từ mọi nguồn. Quy tắc này hoạt động như chiếc cầu chì an toàn cuối cùng, bảo đảm hạ tầng mạng bên trong không bị quá tải khi đối mặt với lưu lượng đột biến.

Rate Limiting Implementation với Redis:

Spring Cloud Gateway sử dụng **Token Bucket algorithm** (Redis-backed) — mỗi người dùng sở hữu một

“bucket” (thùng chứa) thẻ bài (tokens), mỗi yêu cầu tiêu thụ một thẻ bài, và các thẻ bài này được bổ sung theo tỷ lệ `replenishRate` :

```
# application.yml — Rate limiting cho endpoint chấm bài
spring:
  cloud:
    gateway:
      routes:
        - id: kbm-core-rate-limited
          uri: lb://kbm-core
          predicates:
            - Path=/api/core/submissions/**
          filters:
            - StripPrefix=2
            - name: RequestRateLimiter
              args:
                redis-rate-limiter.replenishRate: 5 # 5 requests/giây
                redis-rate-limiter.burstCapacity: 10 # Burst tối đa 10
                redis-rate-limiter.requestedTokens: 1 # 1 token/request
                key-resolver: "#{@userKeyResolver}" # Rate limit theo user

data:
  redis:
    host: localhost
    port: 6379
```

Listing 8.4: Rate limiting configuration cho submission endpoint

Trong cấu hình trên, bộ lọc `RequestRateLimiter` được áp dụng vào route nộp bài tập với các thông số về tỷ lệ phục hồi và dung lượng token. Tuy nhiên, Gateway cần một `KeyResolver` để xác định định danh (khóa) cho từng yêu cầu nhằm cấp phát hạn ngạch phù hợp. Đoạn mã dưới đây triển khai `KeyResolver` dùng `X-User-Id` (do bộ lọc xác thực cung cấp), kết hợp fallback an toàn để tránh `NullPointerException` đối với các API ẩn danh:

```

@Configuration
public class RateLimitConfig {

    @Bean
    public KeyResolver userKeyResolver() {
        return exchange -> {
            // Lấy userId từ header đã được JWT filter inject
            String userId = exchange.getRequest().getHeaders()
                .getFirst("X-User-Id");
            if (userId != null) {
                return Mono.just(userId); // Rate limit per user
            }
            // Fallback: rate limit per IP cho anonymous requests
            String forwardedFor = exchange.getRequest().getHeaders().getFirst("X-Forwarded-For");
            if (forwardedFor != null) {
                return Mono.just(forwardedFor.split(",")[0].trim());
            }
            java.net.InetSocketAddress remoteAddress = exchange.getRequest().getRemoteAddress();
            if (remoteAddress != null && remoteAddress.getAddress() != null) {
                return Mono.just(remoteAddress.getAddress().getHostAddress());
            }
            return Mono.just("anonymous");
        };
    }
}

```

Listing 8.5: KeyResolver an toàn chống NPE — xác định rate limit theo userId

Khi người dùng vượt quá hạn ngạch (limit), Gateway tự động trả về mã lỗi **HTTP 429 Too Many Requests**, qua đó thông báo trực tiếp cho client mà không cần chuyển tiếp yêu cầu xuống các dịch vụ phía sau.

💡 Rate limit cho contest mode

Trong thời gian diễn ra kỳ thi (contest mode), tần suất nộp bài cao hơn bình thường (hàng trăm sinh viên nộp bài cùng lúc). Cần lưu ý các điểm sau: (1) Cấu hình mức rate limit cao hơn cho đường dẫn `/submissions/**` trong kỳ thi; (2) Hoặc sử dụng cơ chế **giới hạn theo từng đường dẫn (per-route rate limit)** dành riêng cho các API thi cử; (3) Sử dụng tính chất nguyên tử (atomic operations) của Redis để đảm bảo đếm chính xác ngay cả khi có lượng lớn yêu cầu đồng thời (concurrent requests).

8.4.5. Logging & Tracing

Gateway là điểm lý tưởng để gắn **correlation ID** — mã định danh duy nhất theo dõi request xuyên suốt hệ thống. Cơ chế hoạt động: Bộ lọc `GlobalFilter` tại Gateway sẽ kiểm tra tiêu đề `X-Correlation-Id`. Nếu tiêu đề này chưa tồn tại, hệ thống sẽ khởi tạo một UUID mới, đính kèm vào request, và truyền xuyên suốt qua tất cả các dịch vụ nội bộ (downstream services). Nhờ đó, khi cần gỡ lỗi (debug), lập trình viên có thể truy xuất log dựa trên correlation ID để theo dõi toàn bộ quá trình (journey) của yêu cầu.

```

@Component
public class CorrelationIdFilter implements GlobalFilter, Ordered {
    @Override
    public Mono<Void> filter(
        ServerWebExchange exchange, GatewayFilterChain chain) {
        String correlationId = exchange.getRequest().getHeaders()
            .getFirst("X-Correlation-Id");

        if (correlationId == null) {
            correlationId = java.util.UUID.randomUUID().toString();
            ServerHttpRequest request = exchange.getRequest().mutate()
                .header("X-Correlation-Id", correlationId)
                .build();
            return chain.filter(
                exchange.mutate().request(request).build()
            );
        }
        return chain.filter(exchange);
    }
    @Override
    public int getOrder() { return -2; }
}

```

Listing 8.6: Correlation ID GlobalFilter

Gateway cũng là vị trí lý tưởng để ghi nhận **access log tập trung**, bao gồm: method, đường dẫn (path), mã trạng thái, độ trễ (latency) và định danh người dùng (từ JWT claims). Thông tin này giúp hệ thống dễ dàng phát hiện endpoint nào có độ trễ bất thường, người dùng nào thường xuyên gặp lỗi, và sự thay đổi trong traffic pattern trước và sau khi deploy. Giải pháp này giúp loại bỏ yêu cầu chèn thêm mã nguồn vào từng dịch vụ riêng lẻ — một **GlobalFilter** duy nhất đủ cho toàn hệ thống.

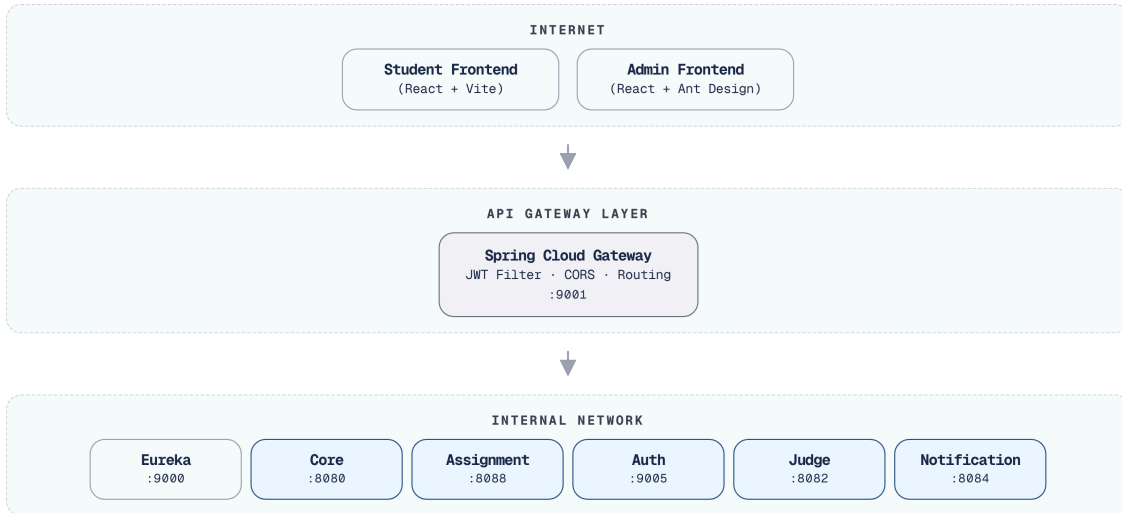
Chi tiết triển khai structured logging, log aggregation, và distributed tracing xem Ch.11 Observability.

8.5. Case Study: KBM

Triển khai kiến trúc Gateway vào hệ thống KBM giống như việc tái cấu trúc luồng giao thông tại cổng chính của trường đại học: giúp tập trung hóa việc kiểm soát, giám sát và điều hướng cho hàng loạt các ứng dụng phía sau.

8.5.1. Kiến trúc tổng thể

Để hiểu rõ cách cấu hình API Gateway trong thực tế, hãy xem xét kiến trúc của hệ thống KBM (8.7). Gateway hoạt động như điểm chạm duy nhất (single entry point), làm nhiệm vụ định tuyến và che giấu sự phức tạp của các dịch vụ mạng nội bộ:



Hình 8.7: Kiến trúc tổng thể KBM – Gateway là single entry point

Hệ thống KBM sở hữu hai loại cổng giao tiếp/bộ định tuyến cần được phân biệt rõ:

Spring Cloud Gateway – dành cho hệ thống LMS lõi: chịu trách nhiệm định tuyến API theo đường dẫn (path), xác thực JWT, cấu hình CORS, giới hạn lưu lượng (rate limiting), và gắn mã theo dõi (correlation ID).

Go hostname-based router – bộ định tuyến dựa trên tên miền (hostname-based router) viết bằng Go dành cho DevOps Lab: thực hiện định tuyến môi trường làm việc (workspace/lab runtime) dựa trên tên miền hoặc tên miền phụ (subdomain) nội bộ, đồng thời tích hợp trực tiếp với k3s/Sysbox ở lớp hạ tầng.

8.5.2. Phân tích configuration

Aspect	Hiện trạng KBM	Best Practice	Gap
Routing	lb:// URIs qua Eureka cho Java services; hostname routing cho DevOps Lab	✓ Đúng theo từng boundary	Cần tài liệu hóa route ownership
JWT Validation	Custom JwtRequestFilter	✓ Đúng (validate tại edge)	—
CORS	allowedOrigins: ""	× Nên restrict	Liệt kê origins cụ thể
Rate Limiting	Không có	× Nên có	Redis + RequestRateLimiter
SSL/TLS	Không ở gateway level	! Tùy deployment	Thường terminate tại NGINX/LB
Correlation ID	Không có	! Nên thêm	GlobalFilter gắn UUID
Path Rewriting	StripPrefix filters	✓ Đúng	—
WebSocket	Route tới notification service	✓ Đúng	—
Health Check	Actuator endpoints	✓ Đúng	—

Bảng 8.1: Phân tích configuration Gateway trong KBM

8.5.3. Vấn đề JWT Version Inconsistency

Một vấn đề cần lưu ý trong KBM: Gateway sử dụng **JJWT 0.11.5** (API mới: `parserBuilder()`) trong khi các dịch vụ khác sử dụng **JJWT 0.9.1** (API cũ: `parser()`). Với HS256 đơn giản, hai phiên bản này tương thích trong các trường hợp cơ bản (happy path). Tuy nhiên, khi tiến hành nâng cấp hoặc bổ sung thuật toán RS256, sự sai lệch phiên bản (version mismatch) có thể dẫn đến tình trạng xác thực không nhất quán (inconsistent validation) — cụ thể là Gateway chấp nhận (accept) yêu cầu nhưng dịch vụ phía sau lại từ chối (reject), hoặc ngược lại.

JWT library version inconsistency

Gateway sử dụng JJWT 0.11.5, trong khi các dịch vụ khác sử dụng 0.9.1. Với thuật toán khóa đối xứng đơn giản (HS256), hai phiên bản này có thể tương thích trong luồng xử lý chuẩn (happy path). Tuy nhiên, khi nâng cấp hoặc bổ sung các tính năng phức tạp (như RS256, token refresh), sự sai lệch phiên bản (version mismatch) sẽ gây ra các lỗi tiềm ẩn khó gỡ lỗi (debug). **Migration path:** (1) Đồng bộ phiên bản JJWT tại tập tin cấu hình cha (parent POM), (2) Đóng gói và tách các hàm xử lý JWT thành một thư viện dùng chung, (3) trong dài hạn: cân nhắc chuyển đổi sang Spring Security OAuth2 Resource Server (hỗ trợ JWT có sẵn, không cần custom JwtUtil).

8.5.4. Đề xuất migration

Phase 1 — Security Hardening (ưu tiên cao, chi phí triển khai thấp):

- Hạn chế chặt chẽ CORS origins (liệt kê các frontend URL cụ thể)
- Thống nhất phiên bản JJWT
- Tài liệu hóa ranh giới (boundary) giữa Spring Cloud Gateway và Go hostname router

Phase 2 — Observability (ưu tiên trung bình):

- Thêm `X-Correlation-Id` GlobalFilter
- Request/response logging tại Gateway

Phase 3 — Protection (ưu tiên trung bình):

- Rate limiting per user (Redis + RequestRateLimiter)
- Giới hạn kích thước payload cho các file upload endpoints

⚠ Sai lầm thường gặp

Đưa business logic vào Gateway

Gateway thực hiện việc biến đổi dữ liệu (data transformation), kiểm tra quy tắc hợp lệ (validation rules), hoặc điều phối các lời gọi (orchestrate calls) giữa các dịch vụ. Hậu quả: Gateway sẽ biến thành một monolith mới – mọi thay đổi về business logic đều đòi hỏi phải triển khai (deploy) lại toàn bộ hệ thống Gateway. **Phòng tránh:** Gateway chỉ nên đảm nhiệm xử lý các cross-cutting concerns (auth, CORS, routing, rate limiting). Business logic thuộc về các services.

Validate JWT ở cả Gateway lẫn services

Mỗi dịch vụ vẫn tự thực hiện quá trình xác thực token JWT mặc dù Gateway đã hoàn tất bước này. Hệ quả là gây ra sự trùng lặp logic, tăng độ trễ (latency - mỗi yêu cầu phải xác thực hai lần), và tiềm ẩn nguy cơ bất nhất quán (inconsistency) khi các dịch vụ sử dụng phiên bản thư viện JWT khác nhau.

Giải pháp phòng tránh: Gateway chịu trách nhiệm xác thực JWT và truyền các thông tin định danh (claims) qua các tiêu đề HTTP đáng tin cậy (như `X-User-Id`). Các dịch vụ hoàn toàn tin tưởng Gateway vì đây là lưu lượng mạng nội bộ (internal traffic) và không tiếp xúc trực tiếp với Internet.

Không có phương án dự phòng (fallback) khi Gateway ngừng hoạt động (SPOF)

Gateway là điểm vào duy nhất (single point of entry) cũng đồng nghĩa là điểm lỗi duy nhất (single point of failure). Hậu quả: Khi Gateway gặp sự cố (crash), **toàn bộ** hệ thống sẽ trở nên gián đoạn và không thể truy cập (unreachable). **Giải pháp phòng tránh:** triển khai Gateway theo mô hình Tính sẵn sàng cao (High Availability - HA) bằng cách chạy nhiều instances sau một Load Balancer (ví dụ: NGINX/ALB). Đồng thời, Gateway không được phép gặp lỗi dây chuyền (Cascading Failure): Nếu hệ thống Redis (sử dụng cho cơ chế Rate Limiting) gặp sự cố ngừng hoạt động, Gateway cần được trang bị Circuit Breaker để cho phép lưu lượng truy cập tiếp tục đi qua (pass-through traffic) thay vì làm đình trệ toàn bộ hệ thống. Ở phía Client, phải implement `Exponential Backoff` khi nhận HTTP 429.

CORS `allowAll` trong production

Cho phép mọi nguồn gốc (origin) gọi API bằng thông tin xác thực của người dùng (user credentials). Hệ quả là dẫn đến rủi ro CSRF, nơi bất kỳ trang web độc hại nào cũng có thể thay mặt người dùng thao tác với API. **Giải pháp phòng tránh:** Liệt kê rõ ràng các nguồn gốc hợp lệ và kiểm thử kỹ lưỡng sự tương tác giữa các nguồn gốc được phép (allowed origins) với thông tin xác thực.

Bỏ qua các cơ chế bảo mật nâng cao tại Gateway

Việc áp dụng các kỹ thuật bảo mật nâng cao giúp Gateway trở thành chốt chặn an toàn:

- **JWKS (JSON Web Key Set):** Gateway không hardcode Public Key tĩnh để giải mã JWT. Thay vào đó, tự động fetch từ endpoint `/.well-known/jwks.json` của Auth Service để cho phép xoay vòng khóa (key rotation) định kỳ, kết hợp cùng bộ đệm nội bộ (In-memory Cache) nhằm giảm tải I/O.
- **Redis Blacklist:** Gateway xác thực JWT stateless, nên cần tích hợp Redis Blacklist để chặn token đã bị thu hồi (ví dụ: sinh viên bị đình chỉ). Để tránh lỗi dây chuyền nếu Redis gặp sự cố, Gateway cần trang bị Fallback/Circuit Breaker.
- **Giới hạn kích thước (Payload Size Limit):** Áp dụng giới hạn cứng để chặn tấn công DoS hoặc payload quá lớn trước khi forward request.
- **Phantom Token Pattern:** Khi Gateway hoạt động như BFF, cần phát hành `HttpOnly Cookie` cho frontend thay vì trả về JWT Bearer Token (ngăn chặn XSS), sau đó tự động chuyển đổi cookie thành JWT khi định tuyến xuống các microservices.

 Interactive Demo

Xem minh họa động tại companion web:

Cơ chế định tuyến của API Gateway – – [api-gateway-routing.html](#)

Service Discovery & Registry – – [service-discovery.html](#)

8.6. Tổng kết

API Gateway giải quyết một trong những thách thức cơ bản nhất khi client tương tác với hệ thống microservices: thay vì phải biết địa chỉ của từng service, client chỉ cần biết một endpoint duy nhất. Gateway đảm nhận việc routing, authentication, CORS, và các cross-cutting concerns – các services phía sau chỉ việc tập trung vào business logic.

Spring Cloud Gateway (WebFlux) là lựa chọn hiện đại cho hệ sinh thái Java – reactive, non-blocking, tích hợp sâu với Spring Cloud (Eureka, Circuit Breaker). Việc cấu hình định tuyến (route configuration) với `lb://` URIs kết hợp tự động với service discovery và load balancing – nhờ đó các services có thể scale mà không cần thay đổi cấu hình tại Gateway.

Việc tập trung xử lý các cross-cutting concerns tại Gateway – như JWT validation, CORS, rate limiting, correlation ID – là khoản đầu tư giúp hệ thống bảo mật, dễ debug, và dễ vận hành hơn. Nguyên tắc cốt lõi: Gateway xử lý các vấn đề hạ tầng (infrastructure concerns), các services xử lý business logic – ranh giới rõ ràng, trách nhiệm rõ ràng.

Phân tích hệ thống KBM cho thấy kiến trúc Gateway (gateway architecture) cơ bản là chính xác (Spring Cloud Gateway + Eureka + JWT cho phân hệ LMS core, Go hostname router cho DevOps Lab), nhưng vẫn còn thiếu rate limiting, CORS mở quá rộng ở một số môi trường, và tồn tại sự sai lệch phiên bản (version inconsistency) của thư viện JWT. Lộ trình chuyển đổi (migration path) cần thực hiện rõ ràng: ưu tiên củng cố bảo mật (hardening) trước (CORS, JWT version), sau đó tăng cường khả năng quan sát (observability) (correlation ID, logging), và cuối cùng là bảo vệ hệ thống (protection) (rate limiting).

Ở Chương 9, chúng ta sẽ tìm hiểu sâu về **bảo mật microservices** – JWT structure, OAuth2 integration, chiến lược dual-secret key rotation, và RBAC trong KBM.

 Bài tập Chương 8

Xem Phụ lục Bài tập để luyện tập:

8-1 – Case Study: Netflix Zuul → Spring Cloud Gateway → Custom Gateway ([L2])

8-2 – ADR: Rate Limiting Strategy (Architecture Decision Record [L2])

8.7. Đọc thêm

Sách tham khảo chính:

- [2a] Chris Richardson, *Microservices Patterns*, 1st Ed. – Ch.8: External API Patterns (API Gateway, BFF)
- [4a] Sam Newman, *Building Microservices* – Ch.4: Integration, Smart Endpoints and Dumb Pipes
- [1] Thomas Erl, *SOA Analysis & Design* – API Gateway trong enterprise SOA context

Sách bổ trợ:

- [2b] Chris Richardson, *Microservices Patterns*, 2nd Ed. – Ch.8: External API Patterns (updated)

5. [5] Hugo Rocha, *Practical Event-Driven MS Architecture* — Ch.9: UI in EDA, BFF pattern

Nguồn trực tuyến:

- Spring Cloud Gateway reference — docs.spring.io/spring-cloud-gateway
- Netflix Zuul → Spring Cloud Gateway migration guide
- OWASP API Security Top 10 — owasp.org/www-project-api-security

Tài liệu tham khảo

- Richardson, C. (2018). *Microservices Patterns: With examples in Java*, 1st Edition. Manning Publications.
- Richardson, C. (2025). *Microservices Patterns: With examples in Java*, 2nd Edition. Manning Publications.